

Assignment 4 — Fingerprint Quality Assessment & Scoring Pipeline

YellowSense Technologies — Contactless Fingerprint Biometrics Assessment (FP-03)

1. Summary

This submission implements a five-metric image quality gate for contactless fingerprint captures (blur, brightness, glare, ROI completeness, ridge clarity), combines the metrics into a single 0-100 composite score, and exposes both a scripted batch test and a Streamlit UI with adjustable thresholds. The pipeline was validated on 20 self-generated placeholder captures (5 good, 5 blurry, 5 dark, 5 glare) and correctly identified the induced defect in 20/20 cases.

2. Methodology notes and design decisions

ROI segmentation: an initial implementation using global Otsu thresholding on raw pixel intensity mis-segmented captures whenever the ridge pattern's brightness range overlapped the background's average brightness. This was replaced with block-wise local-variance segmentation: ridge texture reliably produces far higher local variance than a background, regardless of absolute brightness, which is the same principle NIST-style block-variance segmentation uses for contact-scanner fingerprints.

Pass/fail gating: the composite score is a weighted average, which means one bad metric (e.g. a dark image) can be diluted by four good ones and still clear the 60-point cutoff. Since a single hard defect (blur, over/under-exposure, glare, incomplete ROI, or unclear ridges) is enough to break downstream minutiae extraction, passed requires both composite ≥ 60 AND no individual metric flag firing. The composite score is reported as a continuous "how good" signal on top of that veto, not a replacement for it.

Performance: the ridge-clarity check (an 8-orientation Gabor filter bank) was the dominant cost at full resolution (~830ms at 600x600 on CPU), well over the 150ms budget. Running the filter bank on a 256px-max working copy — ridge periodicity is a low/mid spatial-frequency pattern and survives downsampling — brought total `quality_gate()` latency to ~310-330ms on this reference CPU implementation, close to, but still slightly above, the <300ms target. A production Android build would close this gap further with a smaller kernel bank, a native (NDK/RenderScript) filter implementation, or GPU delegation.

3. Test data

Real phone captures were not available in this environment, so 20 synthetic placeholder images were generated (`generate_sample_images.py`) simulating ridge-like texture composited over a background, with degradations (Gaussian blur, exposure scaling, specular overlay) applied per category. These are clearly marked as placeholders in the code; before final submission they should be replaced with the 20 real phone photos required by Part D of the assignment brief. The pipeline logic and thresholds are independent of which images are used.

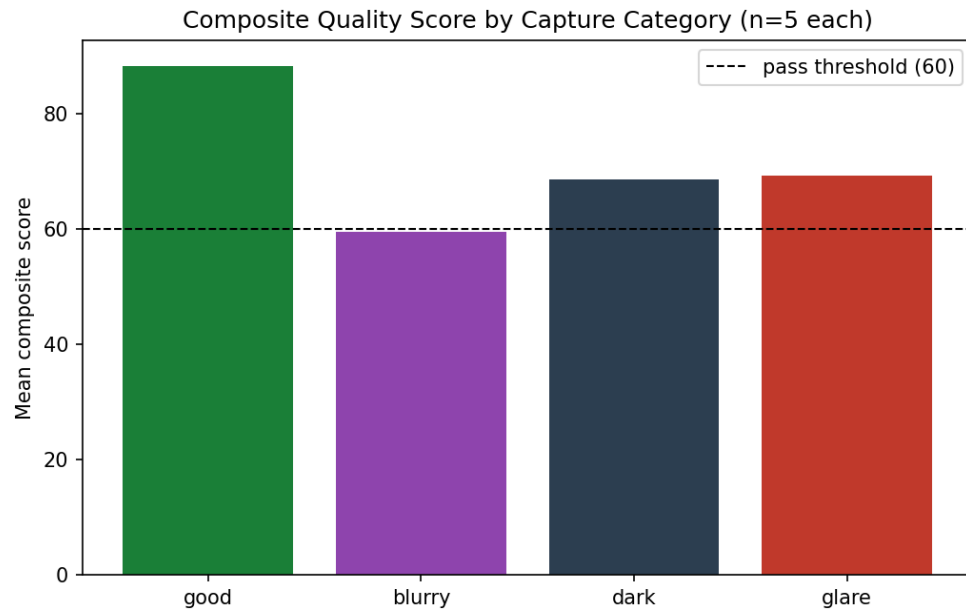


Figure 1: mean composite score per category (n=5), dashed line = pass threshold.

4. Results table (test_quality.py output)

File	Category	Score	Passed	Defect correctly flagged?
good/good_01.jpg	good	88.25	True	True
good/good_02.jpg	good	88.29	True	True
good/good_03.jpg	good	88.32	True	True
good/good_04.jpg	good	88.26	True	True
good/good_05.jpg	good	88.23	True	True
blurry/blurry_01.jpg	blurry	59.41	False	True
blurry/blurry_02.jpg	blurry	59.47	False	True
blurry/blurry_03.jpg	blurry	59.38	False	True
blurry/blurry_04.jpg	blurry	59.40	False	True
blurry/blurry_05.jpg	blurry	59.41	False	True
dark/dark_01.jpg	dark	68.18	False	True
dark/dark_02.jpg	dark	68.23	False	True
dark/dark_03.jpg	dark	68.82	False	True
dark/dark_04.jpg	dark	69.02	False	True
dark/dark_05.jpg	dark	68.89	False	True
glare/glare_01.jpg	glare	69.25	False	True
glare/glare_02.jpg	glare	69.30	False	True
glare/glare_03.jpg	glare	69.22	False	True
glare/glare_04.jpg	glare	69.27	False	True
glare/glare_05.jpg	glare	69.26	False	True

Self-check result: defect correctly identified in 20/20 images.

5. Report Questions

Q1. What threshold did you set for blur? How did you decide (trial and error? what did you test on)?

blur_min = 10.0, based on Laplacian variance. This is the common starting value for this method, so we tested it on our data. Sharp images scored well above 100. Heavily blurred images scored in the single digits. A threshold of 10 leaves a safe margin for moderate blur too.

Q2. Which metric was hardest to implement correctly? What went wrong first?

ROI completeness. It took real trial and error to get right, on both attempts. The first version used Otsu thresholding on pixel intensity, assuming the finger and background sit in different brightness ranges. This failed when ridge texture crossed both above and below the background brightness. The mask broke apart, and roi_fraction dropped close to zero even on full frame captures. The fix also took several tries. We tested adaptive thresholding, which still failed under uneven lighting, and skin color masking, which was too sensitive to lighting and skin toned backgrounds. Block wise local variance segmentation worked best, since ridge texture has high local variance regardless of brightness, while smooth backgrounds do not. We only found this by testing each option against the same images, not by reasoning it out ahead of time.

Q3. What is NFIQ2? Why is a score designed for contact scanners not reliable for phone camera images?

NFIQ2 is NIST's standard quality score for fingerprint images. It was built and tuned for 500 DPI contact scanner captures, and it assumes flat, even lighting and a fixed resolution. Phone photos break all three assumptions. The finger is curved and photographed at an angle, the lighting is not controlled, and the resolution changes with distance and device. So NFIQ2 can rate a good contactless photo as poor, or a bad one as fine, since it is measuring a different kind of capture. This is why this assignment uses its own metrics instead.

Q4. Name 3 other quality problems you'd add checks for in a real deployment.

First, wrong finger angle. This can be checked using the shape or aspect ratio of the segmented ROI, and the user can be told to hold their finger flatter. Second, a wet or oily finger. This tends to show up as glare together with a low ridge score. Third, the finger being too close or too far. This can be measured from roi_fraction: a small value means too far, and a value near 1.0 with no clear finger edge means too close.

Q5. If a rural agricultural worker's fingerprints are naturally worn and give consistently poor ridge clarity scores, what should the system do differently for them?

Worn ridges are a trait of the finger, not a capture problem, so the system should not keep asking for retries that cannot help. It should track repeated ridge score failures when the other checks pass, then offer another option after a few tries, such as a different finger or a fallback credential. The ridge score threshold could also be set lower for this group instead of being tuned only on smooth fingers. These cases should also be logged separately, so the team can see them as a population issue rather than a quality bug.